

Building a Python DSL for MLIR

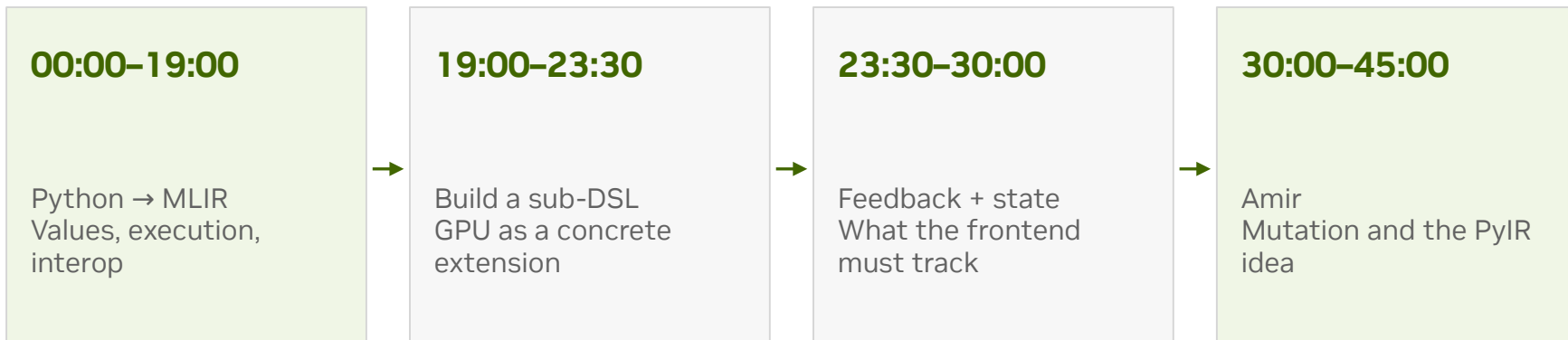
Python is the meta-language. MLIR is the object language.

Reusable core / Sub-DSLs / GenAI feedback

30 minutes + 15 minutes with Amir on mutation and PyIR

The next 45 minutes

A working frontend first; mutation with Amir at minute 30.



Leave with a reusable architecture and a runnable CPU example.

Reuse the Python you already have

Online partial evaluation: execute what is known; emit what remains.

Python runs now

Imports, helpers,
configuration,
and computations over
known values.

MLIR runs later

Typed runtime values,
arithmetic,
regions, loads, and stores.

A reusable base

Share binding, tracing,
diagnostics,
compilation, and invocation.

A DSL author chooses what becomes part of the runtime program.

Three ways to build the frontend

Choose where Python stops and your compiler starts.

AST compiler

Read syntax and lower it.
Own the supported Python semantics.

Proxy tracing

Run Python with value proxies.
Operators emit operations.

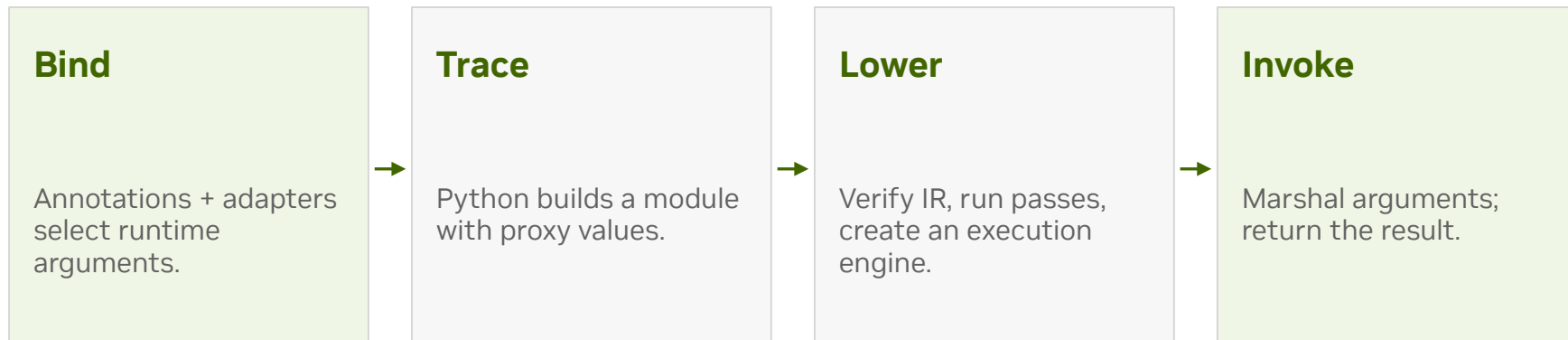
Hybrid

Trace expressions.
Rewrite control flow into callbacks.

This tutorial uses proxies for values and an AST rewrite for regions.

One call, two kinds of execution

The JIT boundary connects ordinary Python to compiled code.



Tracing constructs a program; invocation executes that program.

Binding time is part of the API

An annotation declares a runtime value; plain configuration can specialize.

Definition and call

```
import mlir.dsl as m

@m.jit
def expression(a: m.Int32, b: m.Int32,
               scale) → m.Int32:
    return a + b * scale

expression(1, 2, 3)  # 7
```

Separate runtime data from the Python values that define a specialization.

a and b

Runtime i32 arguments.
Their values do not become constants.

scale

A Python value during tracing.
The generated program contains 3.

Typed values

DSL leaves and adapted buffers
also select runtime data.

An expression becomes operations

Typed Python values are the handles through which tracing emits MLIR.

Python

@m.jit

```
def expression(a: m.Int32, b: m.Int32,  
              scale) → m.Int32:  
    return a + b * scale
```

```
expression(1, 2, 3)
```

Captured MLIR

```
llvm.func @expression_3(  
    %arg0: i32, %arg1: i32) → i32 {  
    %c3_i32 = arith.constant 3 : i32  
    %0 = arith.muli %arg1, %c3_i32 : i32  
    %1 = arith.addi %arg0, %0 : i32  
    llvm.return %1 : i32  
}
```

Runtime operands emit operations; known operands become constants or fold.

Helpers compose during tracing

Use Python abstraction to construct a larger runtime program.

Ordinary Python helper

```
def twice(x):  
    return x + x
```

@m.jit

```
def compute(x: m.Int32) → m.Int32:  
    return twice(x) + 1
```

```
compute(3)  # 7
```

A helper runs now

Its proxy arguments emit operations into the enclosing trace.

Native control flow

Decorate a helper with @m.jit when its control flow must be staged.

Current call boundary

Nested @m.jit bodies inline while the enclosing function traces.

Python call structure need not become runtime call structure.

The same syntax can run now or later

Control-flow dispatch follows the values it sees.

Python syntax	Known values	Runtime values
if predicate	Choose a Python arm	Build scf.if regions
for i in range(n)	Run the Python loop	Build an scf.for
while predicate	Run the Python loop	Build scf.while regions

The loop body is traced once when the loop itself is runtime work.

Watch one local become an SSA carry

The running total crosses the loop boundary explicitly.

Native Python syntax

```
@m.jit
def cumsum(n: m.Int32) → m.Int32:
    acc = m.Int32(0)
    for i in range(n):
        acc += i
    return acc

cumsum(10)  # 45
```

Captured MLIR · SSA names shortened

```
llvm.func @cumsum(%n: i32) → i32 {
    %init = arith.constant 0 : i32
    %lb = arith.constant 0 : i32
    %s = arith.constant 1 : i32
    %r = scf.for %i = %lb to %n step %s
        iter_args(%a = %init) → (i32) : i32 {
            %next = arith.addi %a, %i : i32
            scf.yield %next : i32
        }
    llvm.return %r : i32
}
```

acc has an initial value, a body block argument, and a loop result.

A complete CPU kernel

Use caller-owned buffers and explicit sizes.

Pointer AXPY

```
P = m.Pointer[m.Float32]

@m.jit
def axpy(alpha: m.Float32, x: P, y: P,
        out: P, n: m.Int32):
    for i in range(n):
        out[i] = alpha * x[i] + y[i]

# x, y, out: contiguous float32 NumPy arrays
axpy(2.0, x, y, out, len(x))
```

The generated program uses the existing allocation directly.

One pointer per buffer

NumPy ownership stays with the caller.
The adapter retains the input.

Memory is explicit

Indexing emits getelementptr,
load, and store operations.

A working endpoint

257 elements match NumPy.
The loop becomes scf.for.

Interop is an adapter boundary

Keep framework ownership separate from the DSL value model.

Input	Representation	Role
Contiguous NumPy array	Pointer + retained owner	Current CPU adapter
Contiguous torch.Tensor	Pointer + device kind	Current lazy adapter
DLPack producer	Protocol-managed array view	Alternative adapter design

Choose the runtime ABI deliberately; adapters connect it to Python objects.

Diagnostics are part of the frontend

Preserve Python source locations in both diagnostics and emitted operations.

A Python value crosses phases

@m.jit

```
def cumsum(n: m.Int32) → m.Int32:
    acc = 0
    for i in range(n):
        acc += i
    return acc
```

Diagnostic summary

```
error[PHASE_MUTATE_PYTHON]:
`acc` is a Python value, but it is changed
inside a runtime-controlled region.
```

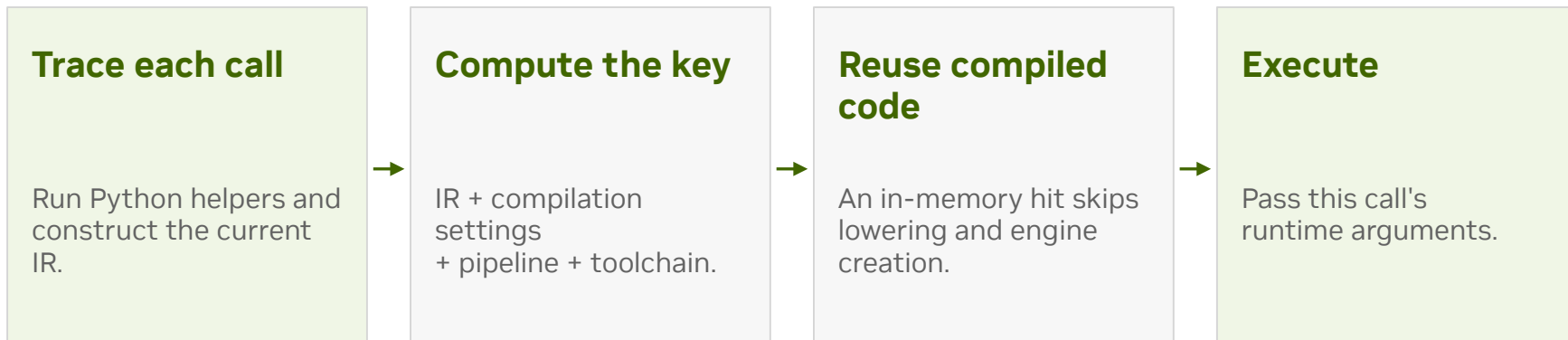
```
21 |         for i in range(n):
    |         ^
```

```
suggestion:
Create `acc` as a runtime value:
    acc = m.Int32(0)
```

MLIR_DSL_DEBUGINFO=1: Python line → FileLineColLoc → operation location.

Cache compilation after tracing

A normal @jit call still observes the current Python state.



A compile cache and a trace cache solve different problems.

Keep the reusable core behind your DSL

Add a plugin when extending a contract; subclass when changing it.

BaseDSL

Binding, preprocessing, tracing, cache, diagnostics, invocation.

Your DSL subclass

Choose entry operations, executors, configuration, and launch behavior.

DialectPlugin

Add dialect registration, operations, lowering passes, and libraries.

Reuse the mechanics; make the domain-specific decisions explicit.

GPU execution is a concrete extension

Separate the device body from the host launch.

Device body

```
P = m.Pointer[m.Float32]
```

@m.kernel

```
def axpy_gpu(a: m.Float32, x: P, y: P,  
            out: P, n: m.Int32):  
    i = m.block_idx()[0] * m.block_dim()[0]  
    i += m.thread_idx()[0]  
    if i < n:  
        out[i] = a * x[i] + y[i]
```

Host launch

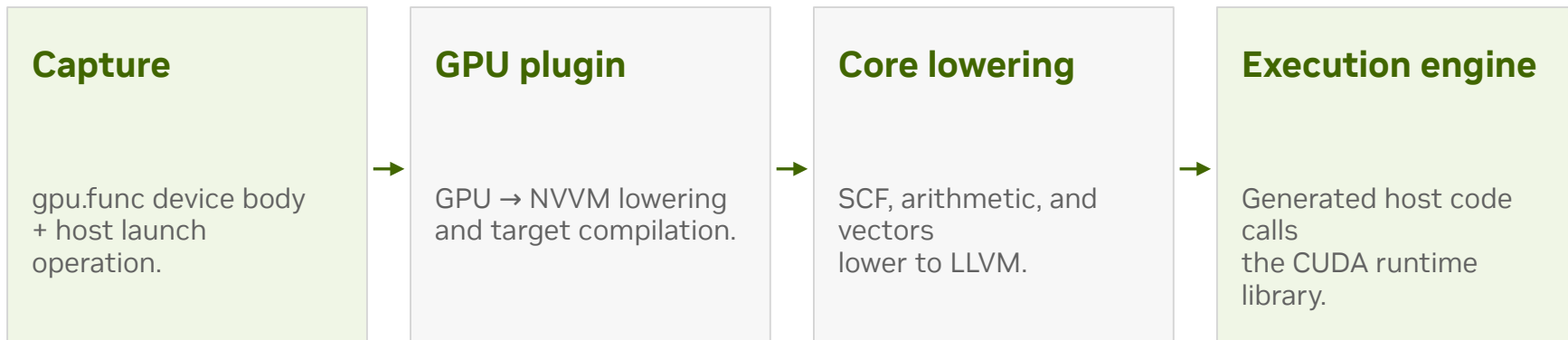
@m.jit

```
def launch(a: m.Float32, x: P, y: P,  
          out: P, n: m.Int32):  
    axpy_gpu(a, x, y, out, n).launch(  
        grid=((n + 255) // 256, 1, 1),  
        block=(256, 1, 1))  
  
# Pass caller-owned device buffers.  
# The framework owns their allocation.
```

The same expression tracing builds a GPU body with an explicit launch.

A plugin supplies the offload path

Compose the target-specific pipeline before the core lowering passes.



The frontend shares its value model; the plugin adds the target path.

Use independent agents as API learners

Give each role a bounded task, fixed resources, and observable checks.

Role	Task	Evidence to keep
ML researcher	CPU sum of squares	Numerical checks + API obstacles
MLIR engineer	Runtime prefix sum	IR carry + yield + results
GPU expert	Vector addition	Tail cases + actual execution
Triton author	Elementwise ReLU	Chosen syntax + diagnostics

Review the code, logs, and failures; treat self-reports as claims to verify.

What the recorded feedback actually shows

Two smoke exercises · older explicit-builder prototype · 21 September 2026

CPU sum of squares

Ten recorded numerical comparisons.
An independent reviewer reran the script.

CUDA ReLU

Empty, singleton, and tail cases.
Recorded framework agreement.

Evidence boundary

Both agents reported first-pass success.
This predates native AST capture.

Keep historical evidence tied to the API version it evaluated.

Make the state transfer visible

Native syntax and explicit builders describe the same loop dataflow.

Explicit carries

```
@m.jit(preprocess=False)
def explicit(n: m.Int32) → m.Int32:
    acc = m.Int32(0)
    for i, a, result in m.for_(
        0, n, 1, [acc]):
        m.yield_([a + i])
    return result
```

AST-assisted syntax

```
@m.jit
def native(n: m.Int32) → m.Int32:
    acc = m.Int32(0)
    for i in range(n):
        acc += i
    return acc

# Both build scf.for with an iter_arg.
# Both return 45 for n = 10.
```

The AST pass identifies writes and threads values through region callbacks.

Which state does the frontend represent?

Value replacement, memory effects, and object identity are different cases.

State change	Current representation	Boundary
acc = acc + i	SSA region carry/result	Runtime leaf with stable type
Fixed-structure container state	Flatten leaves, rebuild structure	Stable shape; unshared identities
out[i] = value	Runtime memory store	Pointer is a reference leaf
Aliased mutable objects	Identity must be preserved	General alias recovery is beyond this model

The next problem is preserving the meaning of mutation across runtime control flow.

From values to mutation

Amir · Mutation and the PyIR idea · 15 minutes

Amir · 15 minutes

AMIR · 30:00–45:00

Amir · Mutation and the PyIR idea

Reserved section · 30:00–45:00

 Reserved for Amir's slides